ClaimShield AI — S4 Solution Design

Team: Walaeddine Ghazouani, Bahaeddine Amara, Omar Jalled, Chedi Kthir, Fedi Mdaini, Maha Ben Mehrez

1. Use Case & Scope

1.1 Problem

Motor insurance claims are still reviewed largely by hand: an adjuster must compare the claim form, the damage photos, the repair invoice and the policy wording. This is slow, costly and inconsistent, and it leaves room for fraud such as photos that contradict the accident description, photos reused from an earlier claim, pre-existing damage presented as new, and inflated repair invoices.

1.2 Solution

ClaimShield AI is an automated claims-review pipeline built in n8n. Each claim is analyzed by three specialized AI agents (photos, invoice, policy), whose findings are combined by a decision agent into an explainable fraud risk score. The score determines whether the claim is approved automatically, sent to a human adjuster, or escalated for fraud investigation.

1.3 Scope

In scope	Out of scope
Private vehicle damage claims (collision, vandalism, glass)	Bodily injury and third-party liability claims
Inputs: claim form, 1–8 damage photos, repair invoice PDF	Video evidence, telematics, police reports
Coverage check against policy terms (RAG)	Legal decisions on disputed coverage
Fraud risk scoring, routing, notifications, audit log	Real payments (a mock payment endpoint is used)
Human approval step for medium-risk claims	Customer portal (claims arrive through a form / API call)

1.4 Inputs, outputs and success criteria

- **Input:** policy ID, incident date and location, free-text description, claimed amount, damage photos, repair invoice PDF.
- **Output:** a decision record (risk score, warning signs found, coverage result with cited policy clauses, route, and short justification) plus the resulting action.
- **Success criteria:** full automation of low-risk claims; on a labeled test set of at least 20 claims, no more than 1 in 20 fraudulent claims auto-approved; every decision explainable; processing under 2 minutes per claim.

2. Team Roles

Role	Responsibilities	Deliverables	Team Members
Team Lead	Planning and follow-up, integration of all workflow parts, action routes (email, Slack, Airtable)	Main orchestration workflow, final demo	Wala Eddine Ghazouani
RAG Engineer	Collecting and chunking policy documents, embeddings, vector database setup, retrieval quality	Policy ingestion workflow, Policy RAG Agent	Chedi Kthiri

Role	Responsibilities	Deliverables	Team Members
Multi-Agent System Architect	Agent design, data formats exchanged between agents, merge logic, fraud scoring rules and thresholds	Fraud Risk & Decision Agent	Omar Jalled
Prompt Engineer & QA Lead	Prompts and JSON output formats for all agents, test claims dataset, evaluation metrics, error handling	Prompt library, test set, evaluation report	Bahaeddine Amara
Multimodal Lead & Documentation Owner	Photo analysis, invoice parsing (OCR), project documentation	Visual Inspection and Invoice Parser Agents, documentation	Fedi Mdaini
Fraud Risk & Decision Agent Lead	Fraud scoring logic, decision thresholds, multi-agent result aggregation, score validation.	Fraud Risk & Decision Agent, scoring documentation.	Maha Ben Mehrez

3. Automation Tool Selection

Selected tool: n8n (self-hosted with Docker during development).

- **AI-native nodes:** OpenAI chat and vision models, embeddings, a Qdrant vector store node and structured output parsers make it possible to build the agents and the RAG pipeline without custom code.
- **Complex logic:** Switch, IF, Merge and sub-workflow nodes support the multi-agent design and multi-branch routing.
- **Human in the loop:** Slack "send and wait for response" pauses the workflow until an adjuster approves or rejects a claim.
- **Privacy and cost:** self-hosting keeps claim data (personal data) on our own infrastructure, with no per-task fees.
- **Deliverables:** workflows export to .json, as required by the course, and can be versioned in Git.

Criterion	n8n	Make	Zapier
Self-hosting	Yes	No	No
Branching and sub-workflows	Strong	Strong	Limited
Native vector store / RAG nodes	Yes	Via HTTP	Limited
Custom code step	JavaScript / Python	Limited	Limited
Pricing	Free self-hosted	Per operation	Per task

4. Solution Architecture

4.1 Overview

The system follows an orchestrator + specialized agents pattern. Simple, deterministic tasks (validation, policy lookup, arithmetic checks, routing) are handled by standard n8n nodes; AI models are used only where interpretation is needed. Each agent is built as a separate sub-workflow so it can be developed and tested independently.

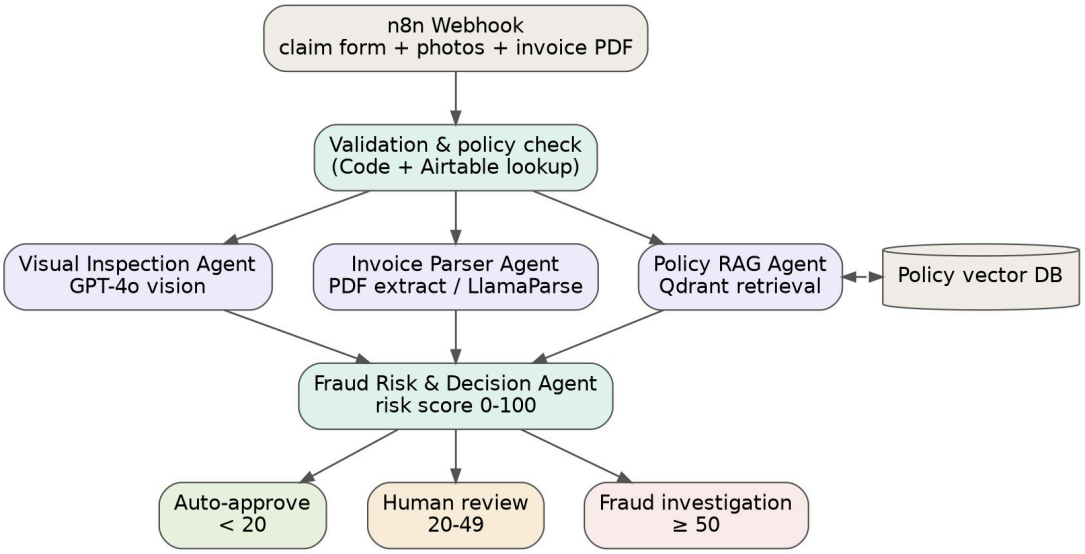


Figure 1 — ClaimShield AI architecture in n8n

4.2 Components

Component	n8n nodes	Role and output
Webhook + validation	Webhook, Code, Airtable	Receives the claim, checks required fields, looks up the policy (active, dates, deductible, coverage limit). Invalid claims are rejected before any AI call.
Visual Inspection Agent	Edit Image, OpenAI (GPT-4o vision), Structured Output Parser	Rates damage severity (1–5), lists damaged parts, checks whether the damage matches the description, flags signs of old damage.
Invoice Parser Agent	Extract From File, HTTP Request (LlamaParse), Information Extractor, Code	Turns the invoice into line items (part, quantity, price, labor) and verifies that totals add up.
Policy RAG Agent	Qdrant Vector Store, OpenAI Embeddings, OpenAI Chat	Retrieves the relevant policy clauses and states whether the damage is covered, citing clause IDs.
Fraud Risk & Decision Agent	Merge, Code, OpenAI (GPT-4o)	Cross-checks the three results, computes the risk score (0–100) and produces the decision record.
Actions	Switch, Gmail, Airtable, Slack, HTTP Request	Sends the approval email, the Slack review request or the investigation ticket; logs everything.

4.3 Data flow

1. **Intake:** the webhook receives the claim and immediately returns a claim ID (HTTP 202), so the client does not wait for processing. The claim is saved in Airtable with status "processing".
2. **Policy check:** policy data is read from Airtable. If the policy is inactive or the incident date is outside the coverage period, the claim is rejected.

3. **Agent analysis:** photos go to the Visual Inspection Agent, the PDF to the Invoice Parser Agent, and the description and policy type to the Policy RAG Agent. Each returns a JSON result in a fixed format.
4. **Cross-modal verification:** a Merge node combines the results; the decision agent compares them, for example invoiced parts that do not appear in the photos, or a described rear impact with front damage visible.
5. **Decision and action:** the risk score is computed and a Switch node routes the claim (Section 4.6). The customer is notified and all agent outputs are stored for audit.

4.4 AI components

- **Multimodal vision (GPT-4o):** all photos of a claim are sent in a single call, resized to reduce cost, so the model can also check that they show the same vehicle. The output follows a fixed JSON schema.
- **Document understanding:** the text layer of the PDF is extracted first; layout-aware OCR (LlamaParse) is used only for scanned invoices, which saves time and cost. An LLM then extracts line items into structured fields.
- **Retrieval-Augmented Generation:** policy documents are split into chunks, embedded (text-embedding-3-small) and stored in Qdrant with metadata (product, section, clause ID). At query time the top 5 chunks, filtered by the customer's policy product, are passed to the model, which must cite the clauses it relies on. Exact figures (deductible, limits) come from the database, not from retrieval.
- **Multi-agent synthesis:** the decision agent combines rule-based checks with an LLM assessment, which keeps the score explainable (Section 4.6).

4.5 Integrations

Service	Purpose	n8n node
OpenAI API	Vision, text extraction, reasoning, embeddings	OpenAI, Embeddings OpenAI
LlamaParse	OCR for scanned invoices	HTTP Request
Qdrant	Vector database for policy clauses	Qdrant Vector Store
Airtable	Policies, claims, decisions, audit log	Airtable
Gmail	Customer notifications	Gmail
Slack	Human review requests and fraud alerts	Slack

4.6 Decision logic

The risk score combines a rule score (maximum 60) with an LLM adjustment (maximum 40) that must be justified in writing. Rules provide objective evidence; the LLM adds nuance but cannot override the rules or trigger an investigation on its own.

Warning sign	Points
Same photo already used in another claim	+40 (automatic investigation)
Visible damage inconsistent with the described accident	+20
Invoice arithmetic does not add up.	+15
Photo date earlier than the incident date	+15
Invoiced part not visible in photos (per part)	+5, max 15
Signs of pre-existing damage	+10
Policy less than 30 days old	+10

3 or more claims in the last 12 months	+10

Route	Condition	Action
Auto-approve	Score below 20 and amount within the auto-approval limit	Approval email, Airtable update, payment request
Human review	Score 20–49, or amount above the limit	Slack approval request to an adjuster
Fraud investigation	Score 50 or above with at least two warning signs, or a reused photo	Investigation ticket and Slack alert

Justification: a single weak signal (such as a new policy) should not block payment; any strong signal justifies a quick human review; a fraud investigation requires at least two independent signs. These starting thresholds are stored in a configuration table and will be calibrated on the labeled test set.

4.7 Example decision record

```
{
  "claim_id": "CLM-2026-00412",
  "risk_score": 47,
  "warning_signs": ["DAMAGE_DESCRIPTION_MISMATCH", "PART_NOT_VISIBLE"],
  "coverage": { "covered": true, "deductible": 500, "clauses": ["C-4.2"] }, "route":
  "HUMAN_REVIEW",
  "justification": "Rear impact described, but front damage visible in photos."
}
```

4.8 Reliability and security

- **Error handling:** automatic retries on AI and API calls; failed claims are marked for manual handling and the team is alerted on Slack.
- **Security:** the webhook requires an authentication header; API keys are stored as n8n credentials; customer messages never mention suspected fraud.
- **Traceability:** every agent result and decision is stored per claim, so any outcome can be explained and replayed.

5. Data Sources & Corpus Collection

To evaluate and demonstrate the end-to-end capabilities of **ClaimShield AI**, the application utilizes structured databases, policy documents, and multimodal test assets across its RAG pipeline and agent evaluation workflows.

5.1 Insurance Policy Corpus (RAG Knowledge Base)

- **Primary Document:** HDFC-Life-Group-Poorna-Suraksha-101N137V02-Policy-Document.pdf
- **Source:** HDFC Life Insurance Company Limited.
- **Format:** Unstructured PDF text and embedded terms.
- **Usage in Pipeline:** Ingested, chunked, and embedded into the **Qdrant Vector Database** using the text-embedding-3-small model.
- **Function:** Serves as the ground-truth policy repository queried by the **Policy RAG Agent** to evaluate claim coverage eligibility and retrieve exact policy clause citations.

5.2 Operational & Claim Databases

- **Airtable Relational Store:**
  - **Policy Database:** Contains active policyholder records, coverage dates,

deductibles, coverage limits, and policy status used by the validation phase.
    - **Claims & Decision Audit Log:** Stores real-time claim submission payloads, agent outputs, cross-modal verification checks, final risk scores, and decision justification logs.

5.3 Test & Evaluation Dataset

- **Labeled Benchmark Dataset:** A test set consisting of at least 20 curated test claims.
- **Multimodal Assets:**
    - **Damage Photos:** 1–8 vehicle damage images per claim (JPG/PNG formats) used by the **Visual Inspection Agent**.
    - **Repair Invoices:** Machine-readable and scanned repair invoice documents (PDF) processed via native PDF extraction and **LlamaParse OCR**.
- **Fraud Scenarios Covered:** Synthesized claims covering duplicate photo reuse, timeline inconsistencies (e.g., photo date prior to incident date), cross-modal description vs. photo mismatches, pre-existing damage, and line-item arithmetic discrepancies.